

ASSIGNMENT-1

Course: Full Stack Web Development
Course Code: 23CM4121
Assignment No.: 1
Student Name: Mohammed Almas
Roll No.: A24126552098
Date: 18-08-2026

Title

JavaScript Inheritance

Aim

To implement and demonstrate different types of inheritance in JavaScript using classes and mixins.

Objectives

1. Understand single inheritance using JavaScript classes.
2. Implement multilevel inheritance.
3. Demonstrate hierarchical inheritance.
4. Demonstrate multiple inheritance using mixins.
5. Execute the program using Node.js and observe the output.

Theory

Inheritance is an object-oriented programming feature in which a class can acquire properties and methods from another class. The assignment demonstrates single, multilevel and hierarchical inheritance with JavaScript classes, and uses mixins to demonstrate multiple inheritance.

Types of Inheritance Implemented

1. Single Inheritance

Dog extends Animal. Dog inherits the eat() method from Animal and defines its own bark() method.

2. Multilevel Inheritance

Bike extends Vehicle and Car extends Bike. The Car object can access methods from all three levels.

3. Hierarchical Inheritance

Admin and Customer both extend User. Both child classes inherit the login() method.

4. Multiple Inheritance using Mixins

Area extends Building and receives methods from the hall and bedRoom objects using Object.assign().

Algorithm / Procedure

Step 1: Create a parent class and define common properties or methods.

Step 2: Create child classes using the extends keyword.

Step 3: Create objects of the child classes.

Step 4: Call inherited and child-specific methods.

Step 5: For hierarchical inheritance, create multiple child classes from one parent.

Step 6: For multiple inheritance, combine methods using mixins and Object.assign().

Step 7: Run the JavaScript file using Node.js and verify the console output.

Program / Code

// Single Inheritance

```
console.log("Single Inheritance");
```

```
class Animal {
    constructor(name) {
        this.name = name;
    }

    eat() {
        console.log(`${this.name} is eating.`);
    }
}

class Dog extends Animal {
    bark() {
        console.log(`${this.name} is barking.`);
    }
}
```

```
const myDog = new Dog("Buddy");
myDog.eat();
myDog.bark();
```

// Multilevel Inheritance

```
console.log("\nMultilevel Inheritance");
```

```
class Vehicle {
    move() {
        console.log("Moving forward");
    }
}
```

Program / Code (continued)

```
    }  
}  
  
class Bike extends C {  
    wheels() {  
        console.log("Bike has 2 Wheels");  
    }  
}  
  
class Car extends C {  
    windows() {  
        console.log("Car has 4 Windows");  
    }  
}  
  
const v = new Cnew);  
v.move();  
v.wheels();  
v.windows();  
  
// Hierarchical Inheritance  
console.log("\nHierarchical Inheritance");  
  
class User {  
    constructor(username) {  
        this.username = username;  
    }  
  
    login() {
```

Program / Code (continued)

```
        console.log(`${this.username} logged in.`);
    }
}

class Admin extends {
    deleteUser() {
        console.log(`${this.username} deleted by admin`);
    }
}

class Customer extends {
    checkout() {
        console.log(`${this.username} checked out successfully.`);
    }
}

const a = new Anvitha("Anvitha");
const c = new Cwomer("Sam");

a.login();
c.login();
a.deleteUser();
c.checkout();

// Multiple Inheritance
console.log("\nMultiple Inheritance");

// Using Mixins
const hall = {
```

Program / Code (continued)

```
    sofa() {
        console.log("Sofa is placed in Hall.");
    },

    tv() {
        console.log("TV is placed in Hall.");
    }
};

const bedRoom = {
    bed() {
        console.log("Bed is in Bedroom.");
    },

    closet() {
        console.log("Closet is in Bedroom.");
    }
};

class Building {
    constructor(houseNo) {
        this.houseNo = houseNo;
    }

    house() {
        console.log(`Building House No: ${this.houseNo}`);
    }
}

class Area extends Building {
```

Program / Code (continued)

```
    constructor(houseNo, area) {  
        super(houseNo);  
        this.area = area;  
    }  
}  
  
// Adding multiple objects' methods to Area  
Object.assign(Area.prototype, hall, bedRoom);  
  
const myArea = new Anew("403", "Kommadi");"403""Kommadi"  
  
myArea.house();  
myArea.sofa();  
myArea.bed();
```

Execution

Command:

`node inheritance.js`

The program creates objects for each inheritance example and prints the inherited and class-specific method results to the Node.js console.

Test Cases

Test Case	Inheritance	Expected Output
TC01	Single	Buddy eating and barking
TC02	Multilevel	Moving, 2 wheels, 4 windows
TC03	Hierarchical	Anvitha and Sam user actions
TC04	Multiple	House, sofa and bed methods

TC05

Execution

Program runs without errors

OUTPUT

01. single-inheritance.png

`node inheritance.js`

Single Inheritance
Buddy is eating.
Buddy is barking.

02. multilevel-inheritance.png

`node inheritance.js`

Multilevel Inheritance
Moving forward
Bike has 2 Wheels
Car has 4 Windows

03. hierarchical-inheritance.png

`node inheritance.js`

Hierarchical Inheritance
Anvitha logged in.
Sam logged in.
Anvitha deleted by admin
Sam checked out successfully.

04. multiple-inheritance.png

`node inheritance.js`

Multiple Inheritance
Building House No: 403
Sofa is placed in Hall.
Bed is in Bedroom.

Result

Thus, the JavaScript inheritance program was implemented successfully. Single, multilevel, hierarchical and multiple inheritance using mixins were demonstrated and the corresponding console outputs were obtained.

Conclusion

The assignment demonstrates how JavaScript classes support inheritance through extends and how mixins can be used to combine methods from multiple objects. The program was executed using Node.js.